

Milestone 1 Report

Smart MCQ Solver Challenge – Dataset Exploration

This report walks through the choices I made while solving the milestone 1 question, and explains why I approached each question the way I did. The notebook itself has the code and outputs, so this is meant to be the reasoning behind it rather than a repeat of the numbers.

Q1. Sum of most frequent and least frequent correct options

For this one I just used `value_counts()` on the answer column since it directly gives a frequency table sorted from highest to lowest. Once I had that, the most frequent and least frequent counts are just the max and min of that series, so `freq.max() + freq.min()` was the simplest way to get there.

Q2. Total unique words in the prompt column

Before counting unique words I lowercased everything and stripped punctuation as told in the question. I used a plain Python set to collect words because sets naturally drop duplicates, so `len(set)` gives the unique word count directly without writing extra logic.

Q3. Meaningful words in the first prompt

I reused the same `clean_text` function from Q2 so the cleaning stays consistent across the notebook, rather than writing a slightly different version here. To remove the “meaningless” filler words I used sklearn's built-in `ENGLISH_STOP_WORDS` as instructed in question. After cleaning, it's just a list comprehension to drop any token that shows up in that stopword set.

Q4. Vocabulary size from the vectorizer

Here I switched from manual cleaning to `TfidfVectorizer` because the next few questions need an actual TF-IDF matrix anyway, so it made sense to build the vectorizer once and reuse it everywhere instead of repeating set-based logic. I combined the prompt and all five option columns into one joined string per row before fitting, since the vocabulary should reflect every word that could possibly show up in a prompt or an option, not just the prompt alone. Used `stop_words="english"` inside the vectorizer itself so I don't have to clean text separately before fitting it and it is mentioned in the question anyway.

Q5. Cosine similarity between the first prompt and option A

Since the vectorizer was already fit on the full vocabulary in Q4, I just reused it here with `transform()` rather than fitting a new one, which keeps the vector space consistent. Cosine similarity is the natural metric for comparing two TF-IDF vectors because it cares about the angle between them, not the raw word counts, so longer or shorter text doesn't unfairly skew the score. Used sklearn's `cosine_similarity` directly instead of writing the dot-product/norm formula by hand since it's a one-liner and already optimized for sparse matrices.

Q6. Best-match accuracy across all five options

Doing this row by row with a Python loop would have been painfully slow on the full dataset, so I vectorized it instead: transform the whole prompt column and each option column into TF-IDF matrices in one shot, then multiply and sum across rows to get similarity scores for all rows at

once. Stacking the five similarity arrays with `column_stack` and taking `argmax` along axis 1 picks the highest-similarity option for every row simultaneously. Comparing predicted letters to the actual answer column and averaging gives accuracy without needing a single explicit loop.

Q9. MAP@3 score of the majority class baseline

This is meant to be a static baseline, so I reused the frequency ranking from Q1, since it already tells us which option is most common, second most common, and so on. I built a small dictionary mapping each of the top three letters to its MAP@3 score (1.0, 0.5, 1/3) following the rule from Q7/Q8, and used `.map()` to apply it across the whole answer column in one go. Anything that isn't one of the top three letters naturally falls outside the mapping, so I filled those with 0 rather than writing a separate if/else check. Taking the mean of that column then gives the overall baseline score directly.

Q10. Average MAP@3 score of the TF-IDF pipeline

I reused the same `sim_scores` from Q6 instead of recomputing similarities, since the underlying TF-IDF comparison doesn't change here, only how I use it does. Instead of just taking the single best match like in Q6, this needed the top three options ranked by similarity, so I used `argsort` (which sorts ascending) and reversed it to get highest-to-lowest, then sliced the first three columns. For scoring, I went with a simple list comprehension that checks where the true answer falls inside each row's top-3 prediction list and gives $1/(\text{rank})$, or 0 if it's missing entirely – this directly mirrors the MAP@3 definition from Q7/Q8 rather than trying to generalize it into a separate function, since we only ever need it for $k=3$ here.

Overall, the approach across the notebook was to keep the early questions simple and pandas-native, then move to a vectorized TF-IDF setup once the questions started needing repeated similarity comparisons – building the vectorizer once in Q4 and reusing it for Q5, Q6, and Q10 avoided redundant work and kept the results consistent.